

Multimodal Video Analysis Application

Multimodal Video Analysis Application

1. Concept Introduction
 - 1.1 What is "Video Analysis"?
 - 1.2 Implementation Principles
2. Code Analysis
 - Key Code
 1. Tool Layer Entry (`largemodel/utis/tools_manager.py`)
 2. Model Interface Layer and Frame Extraction (`largemodel/utis/large_model_interface.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configuring the Offline Large Model
 - 3.1.1 Configuring the LLM Platform (yahboom.yaml)
 - 3.1.2 Configuring the Model Interface (`large_model_interface.yaml`)
 - 3.2 Starting and Testing the Feature (Text Input Mode)
4. Common Problems and Solutions
 - Problem 1: Error message "Video file not found" or "Unable to extract keyframes from video."
 - Problem 2: Analyzing a long video is time-consuming.

1. Concept Introduction

1.1 What is "Video Analysis"?

In the `largemodel` project, the **multimodal video analysis** feature enables a robot to process a video and summarize its core content, describe key events, or answer specific questions about the video in natural language. This allows the robot to leap from understanding only static images to understanding the dynamic and temporal world.

The core tool for this feature is `**analyze_video``. When a user provides a video file and asks a question (such as "Summarize what this video says"), the system invokes this tool to process and analyze the video and return a textual response from the AI.

1.2 Implementation Principles

The core challenge of offline video analysis lies in how to efficiently process video data containing hundreds or thousands of frames. A popular implementation principle is as follows:

1. Keyframe Extraction: First, the system does not process every frame of the video. Instead, it extracts the most representative keyframes through algorithms (such as scene boundary detection or fixed-time sampling). This significantly reduces the amount of data required for processing.
2. Image Encoding: Each extracted keyframe is fed into a visual encoder, similar to visual understanding applications, and converted into a digital vector containing image information.
3. Temporal Information Fusion: This is the most significant difference from single-image understanding. The model needs to understand the temporal order between these keyframes. Typically, a recurrent neural network (RNN) or Transformer model is used to fuse

the vectors of all keyframes to form a "memory" vector that represents the dynamic content of the entire video.

4. Question-Answering and Generation: The user's text question is encoded and then cross-modally fused with this "memory" vector. Finally, the language model generates a summary of the entire video or an answer to a specific question based on this fused information.

Simply put, it condenses a video into a few key images and their sequence, allowing users to understand the entire story like reading a comic strip and answer related questions.

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)

The `analyze_video` function in this file defines the tool's execution flow.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def analyze_video(self, args):
        """
        Analyze video file and provide content description.
        分析视频文件并提供内容描述。

        :param args: Arguments containing video path.
        :return: Dictionary with video description and path.
        """
        self.node.get_logger().info(f"Executing analyze_video() tool with args:
{args}")
        try:
            video_path = args.get("video_path")
            # ... (智能路径回退机制)

            if video_path and os.path.exists(video_path):
                # ... (构建Prompt)

                # Use a fully isolated, one-time context for video analysis to
                ensure a plain text description. / 使用完全隔离的一次性上下文进行视频分析，以确保获得纯文
                本描述。

                simple_context = [{
                    "role": "system",
                    "content": "You are a video description assistant. ..."
                }]

                result = self.node.model_client.infer_with_video(video_path,
                    prompt, message=simple_context)

                # ... (处理结果)
                return {
                    "description": description,
                    "video_path": video_path
                }
            # ... (错误处理)
```

2. Model Interface Layer and Frame Extraction (`largemodel/utils/large_model_interface.py`)

The functions in this file are responsible for processing video files and passing them to the underlying model.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_video(self, video_path, text=None, message=None):
        """Unified video inference interface. / 统一的视频推理接口。"""
        # ... (准备消息)
        try:
            # 根据 self.llm_platform 决定调用哪个具体实现
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
video_path=video_path)
            # ... (其他在线平台的逻辑)
            # ...
            return {'response': response_content, 'messages': self.messages.copy()}

    def _extract_video_frames(self, video_path, max_frames=5):
        """Extract keyframes from a video for analysis. / 从视频中提取关键帧用于分析。"""
        try:
            import cv2
            # ... (视频读取和帧间隔计算)
            while extracted_count < max_frames:
                # ... (循环读取视频帧)
                if frame_count % frame_interval == 0:
                    # ... (将帧保存为临时图片)
                    frame_base64 = self.encode_file_to_base64(temp_path)
                    frame_images.append(frame_base64)
                # ...
            return frame_images
        # ... (异常处理)
```

Code Analysis

The implementation of video analysis is more complex than image analysis. It requires a key preprocessing step at the model interface layer: frame extraction.

1. Tools Layer (`tools_manager.py`):

- The `analyze_video` function is the entry point for video analysis. Its responsibilities are clear: accept a video file path and construct a prompt to request a description of the video content.
- It initiates the analysis process by calling the `self.node.model_client.infer_with_video` method. Like the visual understanding tool, it is completely agnostic about the underlying model details and only handles passing the "video file" and "analysis instructions."

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_video` function is the scheduler that connects the upper-level tools with the underlying model. It dispatches tasks to the corresponding implementation functions

based on the platform configuration (`self.llm_platform`).

- Unlike processing single images, processing videos requires an additional step. The `_extract_video_frames` method demonstrates the general logic for implementing this step: it uses the `cv2` library to read the video and extract several keyframes (by default, five).
- Each extracted frame is treated as a separate image and typically encoded as a Base64 string.
- Ultimately, a request containing multiple frame image data and analysis instructions is sent to the main model. The model performs a comprehensive analysis of these consecutive images to generate a description of the entire video content.
- This "video-to-multiple-images" preprocessing is completely encapsulated in the model interface layer and is transparent to the tool layer.

In summary, the general process of video analysis is: `ToolsManager` initiates an analysis request -> `model_interface` intercepts the request and calls `_extract_video_frames` to decompose the video file into multiple keyframe images -> `model_interface` sends these images, along with analysis instructions, to the corresponding model platform according to the configuration -> the model returns a comprehensive description of the video -> the results are finally returned to `ToolsManager`. This design ensures the stability and versatility of upper-layer applications.

3. Practical Operations

3.1 Configuring the Offline Large Model

3.1.1 Configuring the LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its primary language model.

1. Open the file in the terminal:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. Modify/confirm `llm_platform`:

```
model_service:                                #模型服务器节点参数 Model server node
parameters
  ros__parameters:
    language: 'en'                             #大模型接口语言 Large Model Interface
  Language
    useolinetts: False                         #文字模式下此项无效，可忽略 This item is
invalid in text mode and can be ignored

    # 大模型配置 Large model configuration
    llm_platform: 'ollama'                     # 关键：确保这里是 'ollama' Key: Make
sure it's 'ollama'
    regional_setting : "international"
```

3.1.2 Configuring the Model Interface (`large_model_interface.yaml`)

This file defines which visual model to use when the `ollama` platform is selected.

1. Open the file in a terminal.

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2. Find the configuration related to ollama

```
#.....  
## 离线大模型 (Offline Large Language Models)  
# Ollama Configuration  
ollama_host: "http://localhost:11434" # Ollama server address  
ollama_model: "llava" # Key: Change this to the multimodal model you  
downloaded, such as "llava"  
#.....
```

Note: Please ensure that the model specified in the configuration parameters (e.g., `llava`) can handle multimodal input.

3.2 Starting and Testing the Feature (Text Input Mode)

Note: Due to performance limitations, the Jetson Orin Nano 4GB may not perform well. To experience this feature, please refer to the corresponding section in <Online Large Model (Text Interaction)>

1. **Prepare video files:**

Place a video file to test in the following path:

`/home/jetson/yahboom_ws/src/largemodel/resources_file/analyze_video`

Then name the video `test_video.mp4`

2. **Start the `largemodel` main program (in text mode):**

Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
```

3. **Send text command:**

Open another terminal again and run the following command,

```
ros2 run text_chat text_chat
```

Then start typing text: "Analyze this video."

4. **Observation results:**

In the first terminal running the main program, you will see log output showing that the system received the command, called the `analyze_video` tool, extracted the keyframes, and finally printed out the AI's summary of the video content.

4. Common Problems and Solutions

Problem 1: Error message "Video file not found" or "Unable to extract keyframes from video."

Solution:

1. **Check the path:** Ensure you have placed the video file in the specified path, named it `test_video.mp4`, and have read permissions.
2. **Video Format/Encoding:** Ensure the video file is not corrupted and is in `.mp4` format.

Problem 2: Analyzing a long video is time-consuming.

Solution:

1. **Use a lighter model:** If hardware resources are limited, switch to a video understanding model with fewer parameters.